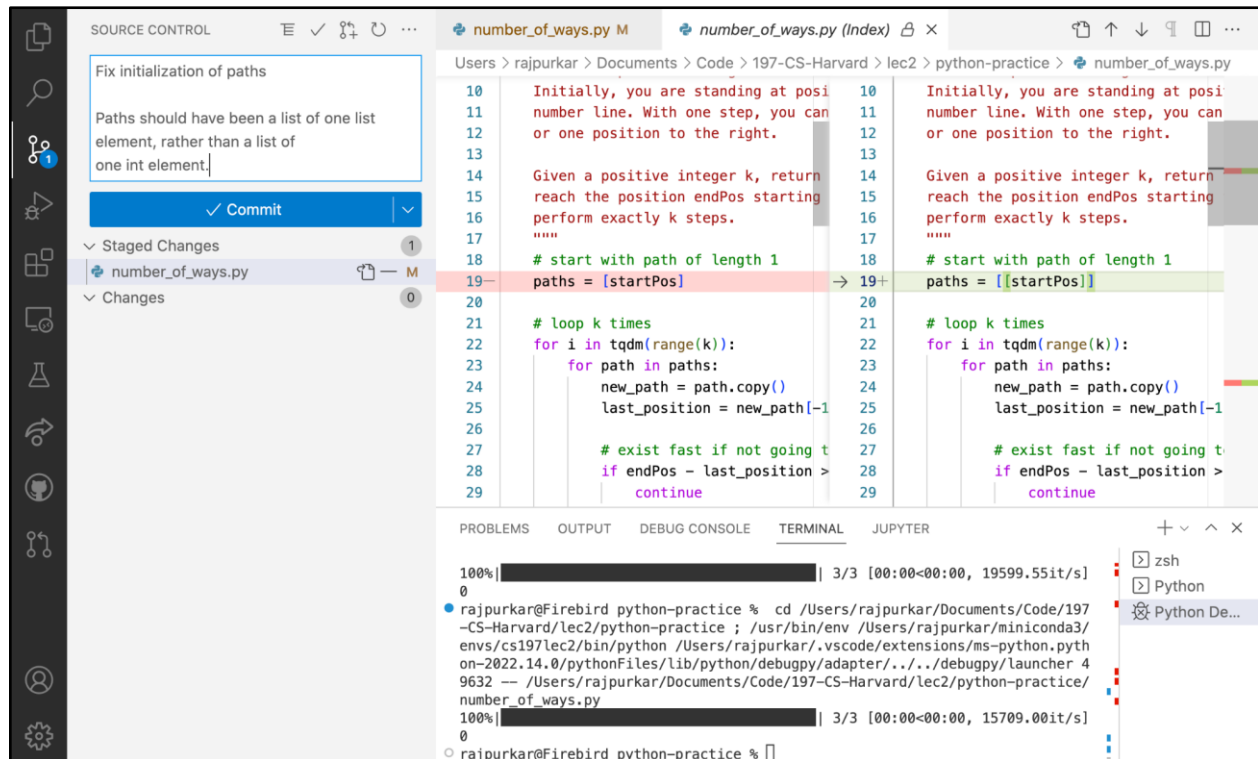


Breakpoints and Logpoints

It looks like the code ran – hurrah! We have solved the attribute error. At this point, we want to git commit the change we made. So we can go ahead and do that.



We shouldn't be too happy yet though. We've gotten the code to run, but is the output what we expect?

From our test docstring itself, it doesn't look like it. We should have returned 3, not 0! So what happened? Let's go back to the debugger.

Breakpoint

To understand what's going on, I am going to see what 'paths' looks like after we have finished populating it. Rather than add a print statement there, we're going to add a breakpoint.

Now we hit run and pause.

You can see that execution has stopped and on the right that we get to see into the paths variable. In addition, we have the debug toolbar which has appeared on top.

What do the colors and shapes in the editor margin mean? From

https://code.visualstudio.com/docs/editor/debugging#_breakpoints:

Breakpoints can be toggled by clicking on the editor margin or using F9 on the current line. Finer breakpoint control (enable/disable/reapply) can be done in the Run and Debug view's BREAKPOINTS section.

- Breakpoints in the editor margin are normally shown as red filled circles.
- Disabled breakpoints have a filled gray circle.
- When a debugging session starts, breakpoints that cannot be registered with the debugger change to a gray hollow circle. The same might happen if the source is edited while a debug session without live-edit support is running.

The debug toolbar has 5 options:

- Continue / Pause F5
- Step Over F12
- Step Into F11
- Step Out ⇧F11
- Restart ⇧⌘F5
- Stop ⇧F5

Why google them when you can simply ask GPT3?

A little more [detail](#) on the difference between Step Over, Into, and Step Out.

- If the current line contains a function call, Step Over runs the code and then suspends execution at the first line of code after the called function returns.
- On a nested function call, Step Into steps into the most deeply nested function. For example, if you use Step Into on a call like `Func1(Func2())`, the debugger steps into the function `Func2`.
- Step Out continues running code and suspends execution when the current function returns. The debugger skips through the current function.

Back to our debugger output; something is suspicious. Our paths should only get longer; how is it that they're getting shorter? Before stepping through the whole function, I'd like to know what the variable 'paths' looks like at the end of every iteration.

Logpoint

I could do this by manually inspecting paths by setting a breakpoint within the outer for loop; but it's a good opportunity for us to learn about another debugging tool – logpoints. See

<https://code.visualstudio.com/blogs/2018/07/12/introducing-logpoints-and-auto-attach>

A Logpoint is a breakpoint variant that does not "break" into the debugger but instead logs a message to the console.

Logpoints allows you to "inject" on-demand logging statements into your application logic, just like if you had added logging statements into your application before starting it. Logpoints are injected at execution time and not persisted in the source code, so you don't have to plan ahead but can inject Logpoints as you need them. Another nice benefit is that you don't have to worry about cleaning up your source code after you are finished debugging.

Here's a good example GIF of how to use logpoints:

Back to our code. We're going to add a logpoint before we add the `new_path_left` and `new_path_right`. Here we will print the loop counter `i`, `paths`, `new_path_left`, `new_path_right` in that loop iteration.

Now we hit restart (notice the editor margin will now have a red diamond corresponding to our logpoint:

We can now open the debug console to find the logging.

Are you impressed? We've been able to add logging without making our code clunky with print statements everywhere.

Exercise

Take a few minutes to think about what the expected outputs here should have been. How many issues do you see in the above output?

I can see issues here:

1. Left and Right are the same in the first iteration; we would have wanted it to be Left: [1, 0], and Right: [1, 2].
2. `i = 0` repeats thrice, but we should be at `i = 0` just once, because the length of paths when we start should be 1.
3. All paths towards the end of an iteration should be the same length, not different lengths. We're not removing the paths.
4. `i = 1` is where our iteration stops. We should be getting to `i` of 2.

My workflow is going to be the following: I am going to fix each issue by fixing the lines of code corresponding to it, rerun to see the output, and if solved, commit my change to git.

1. Left and Right are the same in the first iteration; we would have wanted it to be Left: [1, 0], and Right: [1, 2].

We should be able to fix this by fixing the line so that the minus is a plus:

```
new_path_right = new_path + [last_position - 1]
```

Let's rerun now:

Woah! We have a program that keeps logging: it's not stopping now, so we're going to need to hit the pause button. But we have the clue we need: we're always at iteration 0! Why would paths never be ending?

Pro tip: It's tempting to get started at solving the next problem, but you should instead take time to commit the fix you made (we're not going to show this). Then we come back to fixing the next problem!

We're making a classic python error: modifying a list when iterating over it. We're going to implement the following solution:

This takes care of both (1) only iterating over the subset of the list that exists at the start of the iteration, and (2) removing the path of interest from the list.

We can now debug this code:

The good news is: all of our previous 4 errors are now no longer! However, the output isn't quite what we expect. We see that the [1,0] branch has been lost even though we could do [1,0,1,2] in the next step. It could be something to do with our 'continue' statement. We're going to set a breakpoint there:

We're going to add both sides of the condition to the 'watch list': here, variables and expressions can be evaluated and watched in the Run and Debug view's WATCH section.

Now that we have added these expressions to the watch list, we can see that the condition is not quite right.

We can fix the error: the Right Hand Side of the expression should be 'k - i', not 'k - i - 1'. Let's fix it, disable the breakpoint, and rerun debug.

We see something good here! In the inner loop, we're running through the `i=0` iteration once, the `i=1` iteration 2 times, and the `i=2` iteration 3 times. Importantly, at the end, we're ending up with 6 possible paths, 3 of which end in 2. That sounds right.

Rather than debug, why don't we hit 'run'?

Another error, but at least we've fixed the last one; we should commit that change. Back to the above error, let's look at what's happening in code.

That's a really easy error to fix! I'm surprised we didn't find it before? Could we have caught that even before execution? Great questions, and the answer lies in linting.

Linting

At this point, we're going to get pretty used to just asking GPT-4 for definitions.

How do we enable a linter to do work within an editor? VSCode has great linting integration:

<https://code.visualstudio.com/docs/python/linting>

Linting can detect "use of an uninitialized or undefined variable, calls to undefined functions, missing parentheses, and even more subtle issues such as attempting to redefine built-in types or functions."

Let's use command palette to enable linting. Let's first use flake8 linting (you may have to use the prompt to complete installation):

We're now going to be able to see a squiggly line underneath `new_ways`, and a problems status (Keyboard Shortcut: `⇧⌘M`).

We can fix these issues now, and commit the changes. After that we can try out the pylint linter as well. This might pick up other style recommendations for us.

It does. I will show one fix here, and leave the rest as an exercise for the reader. Let's say we wanted to rename `startPos` to `start_pos` to make it conform to `snake_case` naming. We can use a VSCode Refactor functionality called Renaming

(https://code.visualstudio.com/docs/editor/refactoring#_rename-symbol):